AMAL JYOTHI COLLEGE OF ENGINEERING KANJIRAPPALLY

B. TECH - COMPUTER SCIENCE AND ENGINEERING

BTCS2023-27-S6: 23CST362-PROGRAMMING IN PYTHON-Project (Batch – I)

Project Title:
NextMovie– Movie Recommendation System

Submitted by

ANNU ZACHARIAS

AJC23CS060

S6 CSE A BATCH

58

INDEX

1.Abstract…….3

2.Description……..…4

3.Libraries and tools……5

4.System design and Workflow ……7

5.Implementation……9

6.Results and Outputs…….11

7.Conclusion and Scope……13

ABSTRACT: NextMovie – Movie Recommendation System

Recommendation systems help users discover relevant content from large collections of data. This project presents a web-based Movie Recommendation System called **NextMovie**, developed using Python and Flask. The system recommends movies based on the language and genre selected by the user.

The application retrieves movie data using the TMDb API and displays movie information such as title, rating, release date, and overview. Users can also view additional details such as cast members and trailer links for each movie.

The web interface is built using HTML, CSS, and JavaScript, providing a colorful and user-friendly experience. This project demonstrates the integration of APIs, web frameworks, and frontend technologies to create an interactive movie recommendation platform.

DESCRIPTION

In today's digital era, movies are produced in large numbers across different languages and genres. With the growth of online streaming platforms and digital movie databases, users often face difficulty finding movies that match their preferences. Searching manually through large collections of movies can be time-consuming and confusing. Therefore, movie recommendation systems play an important role in helping users quickly discover movies that suit their interests.

A movie recommendation system is designed to suggest movies based on certain user preferences such as genre, language, ratings, or previous viewing history. These systems help reduce the time users spend searching for content and instead provide filtered suggestions that are more relevant. Many modern entertainment platforms use recommendation systems to improve the user experience and help audiences explore new content.

The **NextMovie** project is a web-based movie recommendation system developed to help users easily discover movies based on their interests. The system allows users to select a preferred movie language and choose one or more genres. Based on these selections, the application retrieves a list of movies that match the chosen filters and displays them to the user. This makes it easier for users to explore movies that align with their preferences.

The system is developed using Python with the Flask web framework to handle the backend operations. Movie information is retrieved from the TMDb (The Movie Database) API, which provides access to a large collection of movie data including titles, ratings, release dates, descriptions, cast information, and trailers. The frontend of the application is designed using HTML, CSS, and JavaScript to create an interactive and visually appealing interface.

When a user selects the desired language and genres and submits the form, the system sends a request to the TMDb API to fetch relevant movie data. The recommended movies are then displayed on the webpage along with important details such as title, rating, release date, and overview. Users can also click the **Show Details** button to view additional information such as the main cast and a trailer link for the movie.

LIBRARIES AND TOOLS

The development of the NextMovie Movie Recommendation System involves the use of several programming libraries and tools that support both backend development and frontend user interaction. These tools help the system retrieve movie data from an external database, process user input, and present the results in a structured and user-friendly interface.

1. **Python**:
   Python is the primary programming language used for developing the backend of the application. It is widely used in web development due to its simplicity, readability, and extensive library support. In this project, Python is responsible for handling the core logic of the application, processing user inputs such as language and genre selections, and managing communication between the web interface and the external movie database.

2. **Flask**:
   Flask is a lightweight web framework used to build the web application. It provides the structure needed to create web routes, handle HTTP requests, and render HTML templates. Flask acts as the bridge between the frontend interface and the backend logic. When users submit their movie preferences, Flask processes the request and retrieves the relevant movie data from the external API.

3. **Request Library**:
   The Requests library is used to send HTTP requests to external services. In this project, it is used to communicate with the movie database API and retrieve movie information in JSON format. The library allows the system to easily fetch movie data such as titles, ratings, release dates, and descriptions.

4. **TMDB**:
   The TMDb API is used as the main data source for the system. It provides access to a large and continuously updated database of movies. Through the API, the system retrieves information such as movie titles, ratings, release dates, overviews, cast members, and trailer links. This enables the

application to provide real-time movie recommendations based on the user's selected language and genres.

5. **HTML**:
   HTML (HyperText Markup Language) is used to create the structure of the web pages. It defines the layout of the user interface, including the language selection dropdown, genre checkboxes, and the movie recommendation display area.

6. **CSS**:
   CSS (Cascading Style Sheets) is used to design and style the website. It helps create a visually appealing and colorful interface with organized layouts, movie cards, and responsive elements that enhance the overall user experience.

7. **JavaScript**:
   JavaScript is used to add interactivity to the website. It allows the application to dynamically load additional movie information, such as cast details and trailer links, when the user clicks the Show Details button. This improves the performance of the system by loading additional information only when needed.

SYSTEM DESIGN AND WORKFLOW

1. System Design (Architecture)

This project follows a **Modular Web Application Architecture**, where the system is divided into different components that handle specific tasks instead of writing all logic in a single script. Each module performs a dedicated function to ensure the system works efficiently.

The movie recommendation system follows a modular web application architecture where the system is divided into different components. Each component performs a specific task, making the system organized, easy to maintain, and simple to expand in the future.

This layer allows users to interact with the system. It is built using HTML and CSS and includes the webpage layout, a genre selection dropdown, and the section where recommended movies are displayed.

The Flask server acts as the main controller of the application. It receives user input from the webpage, processes the request, and sends the results back to the frontend.

This module contains the core logic of the system. It processes the selected genre and filters the dataset to find movies that match the user's preference.

This component manages the movie dataset stored in a CSV file. It loads the data and allows the system to search and filter movie records efficiently.

This layer displays the final recommendations on the webpage. The recommended movies are presented in a clear list format for the user.

2. The Workflow (Logical Process)

Stage A: System Initialization

When the application starts, the Flask server begins running and prepares the system to handle user requests. The movie dataset stored in the CSV file is loaded so the system can access the data quickly when needed. The HTML and CSS files used for the webpage layout are also initialized.

Stage B: User Interaction

The user opens the website and selects a movie genre from the dropdown menu. After choosing the genre, the user clicks the **Get Recommendations** button, which sends the selected input to the Flask backend.

Stage C: Data Processing

The backend receives the user's input and processes the request. The system searches the dataset and filters the movies that match the selected genre

Stage D: Result Generation

After filtering the dataset, the system collects the matching movie titles and prepares them to be displayed on the webpage.

Stage E: Output Display

Finally, the recommended movies are shown on the webpage in a list format, allowing the user to easily view the suggestions.

IMPLEMENTATION

The Movie Recommendation System is implemented using backend programming and web technologies to create a simple and interactive web application. The system integrates an external movie database API with a user-friendly interface so that users can easily obtain movie recommendations based on their preferred genre.

The movie information used in the system is retrieved using the **TMDb (The Movie Database) API**. Instead of storing movie data locally, the application sends requests to the API and receives movie information such as titles and genres. This allows the system to access a large and updated movie database without maintaining a dataset.

When the application starts, the **Flask web server** runs the backend program and prepares the system to accept user requests. The HTML webpage is then displayed in the browser, where the user can select a movie genre from a dropdown menu.

Once the user selects a genre and submits the form, the selected input is sent from the webpage to the Flask backend. The backend then uses the **API key** to send a request to the TMDb API and fetch movies related to the selected genre.

After the API returns the movie data, the system processes the response and extracts the movie titles. These movie titles are then sent back to the Flask application for display.

Flask uses **HTML templates** to dynamically display the results on the webpage. The recommended movies appear as a list below the input section so that users can easily view the suggestions. **CSS styling** is used to improve the appearance of the webpage by adding layout structure, colors, and fonts.

The project is organized into multiple files to keep the code structured and easy to maintain. Each file is responsible for a specific part of the system.

Main Project Files:

app.py
This file contains the main Flask application. It runs the web server, handles user requests, and connects the frontend interface with the TMDb API.

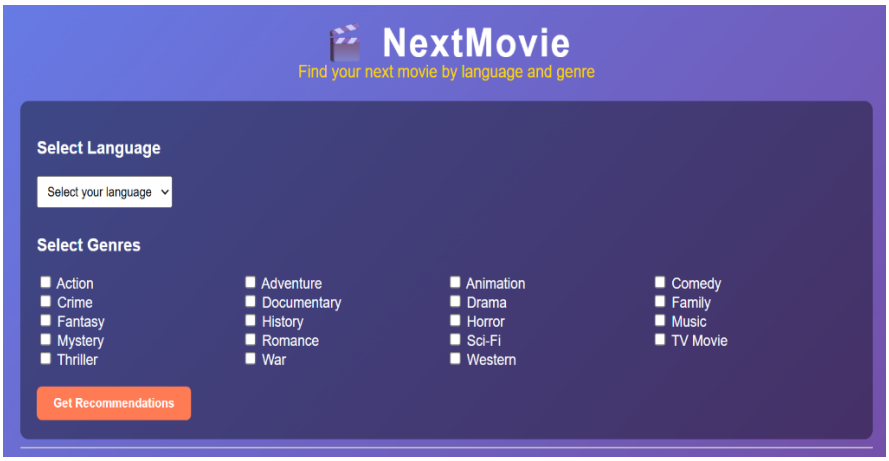
index.html
This file defines the structure of the website interface. It includes the genre selection dropdown menu, input form, and the section where recommended movies are displayed.

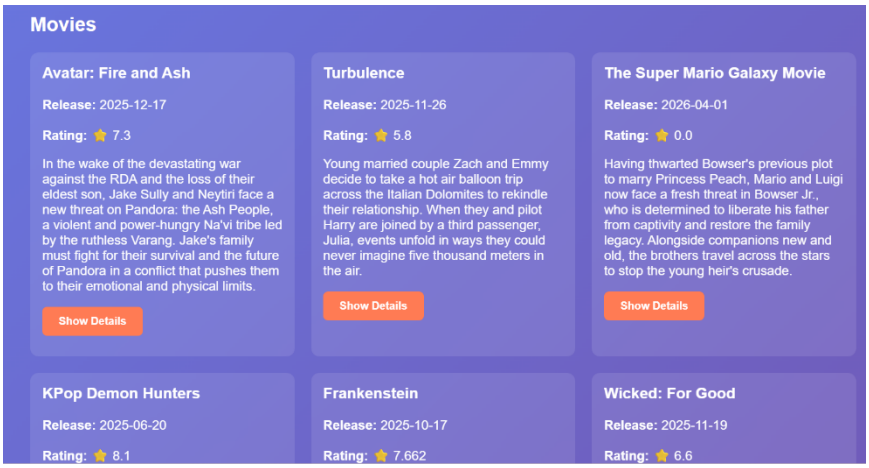
style.css
This file contains the styling for the webpage. It controls the layout, colors, fonts, and overall design of the website.

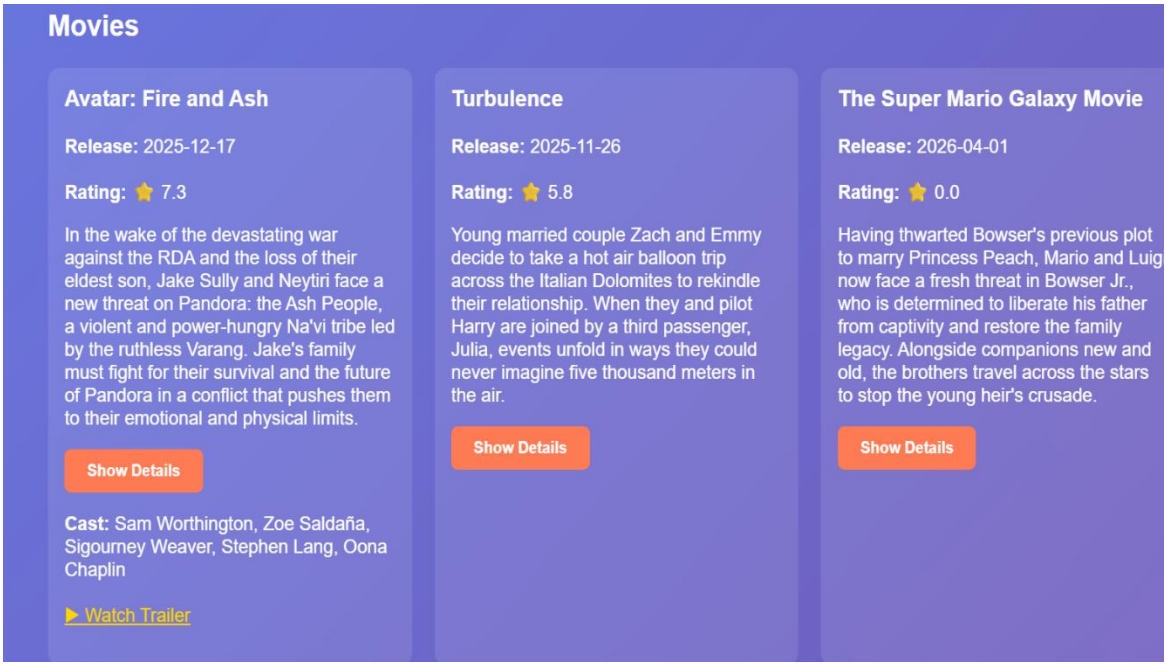
By organizing the project into multiple files and integrating the TMDb API with a web interface, the system becomes structured, interactive, and capable of providing real-time movie recommendations.

RESULTS AND SCREENSHOTS

The system successfully recommends movies based on the genre selected by the user. The website interface allows users to interact with the system easily and receive recommendations instantly.






Movies
Avatar: Fire and Ash
Release: 2025-12-17
Rating: 7.3
In the wake of the devastating war against the RDA and the loss of their eldest son, Jake Sully and Neytiri face a new threat on Pandora: the Ash People, a violent and power-hungry Na'vi tribe led by the ruthless Varang. Jake's family must fight for their survival and the future of Pandora in a conflict that pushes them to their emotional and physical limits.
Show Details
Cast: Sam Worthington, Zoe Saldaña, Sigourney Weaver, Stephen Lang, Oona Chaplin
▶ Watch Trailer
Turbulence
Release: 2025-11-26
Rating: 5.8
Young married couple Zach and Emmy decide to take a hot air balloon trip across the Italian Dolomites to rekindle their relationship. When they and pilot Harry are joined by a third passenger, Julia, events unfold in ways they could never imagine five thousand meters in the air.
Show Details
The Super Mario Galaxy Movie
Release: 2026-04-01
Rating: 0.0
Having thwarted Bowser's previous plot to marry Princess Peach, Mario and Luig now face a fresh threat in Bowser Jr., who is determined to liberate his father from captivity and restore the family legacy. Alongside companions new and old, the brothers travel across the stars to stop the young heir's crusade.
Show Details

CONCLUSION AND SCOPE

The Movie Recommendation System was successfully developed as a simple web application that allows users to discover movies based on their preferred genre. The system provides an easy-to-use interface where users can select a genre and instantly receive movie recommendations. By using **Flask for the backend**, **HTML and CSS for the frontend**, and the **TMDb API** for retrieving movie data, the application is able to provide relevant and updated movie suggestions.

This project demonstrates the practical use of web development technologies and API integration to build an interactive application. The modular design of the system makes it organized and easy to maintain, while also allowing future improvements such as adding more filters or recommendation features. Overall, the project shows how external data sources and web technologies can be combined to create an efficient and user-friendly movie recommendation system.

SCOPE

The scope of the Movie Recommendation System is to provide users with a simple and efficient way to discover movies based on their preferred genre. The system allows users to select a genre and receive movie recommendations by retrieving data from the **The Movie Database (TMDb) API**. This helps users quickly find movies without manually searching through large movie lists.

In the future, the system can be expanded by adding more features such as filtering movies by rating, release year, or language. The application can also be improved by including additional recommendation techniques, displaying movie posters and descriptions, and supporting more categories. These improvements would make the system more advanced and provide a better user experience.